scripts/pre_build/check_cm_killpg_pgid_guard.sh â€” CM-KILLPG-PGID-GUARD static pre-build gate

Revision: 1 **Last modified:** 2026-08-19T00:00:00Z **Status:** active **Item:** BOB-126 follow-up (Â§11.4.238 discovery-channel-escape closure â€” a new automated check for a defect class first found out-of-band)

Purpose

`scripts/pre_build/check_cm_killpg_pgid_guard.sh` statically detects calls that send a signal to a **negative pid** or an entire **process group** (Python `os.killpg(pgid, sig)`, `os.kill(-pid, sig)`; bash `killpg`, an explicit `kill -SIG` invocation) that are **not** preceded, within the 10 lines above the call, by a guard proving the target identifier is a real, positive process id:

```
isinstance(<ident>, int) and <ident> > 1
```

on the **same** identifier that reaches the dangerous call.

The defect class this closes (BOB-126)

`os.killpg(1, SIGKILL)` is, under glibc, **identical** to `kill(-1, SIGKILL)` â€” a broadcast kill of every process the callerâ€™s UID owns. A test double whose `.pid` attribute is an unconfigured mock object (`MagicMock.__int__` defaults to `1`) silently reaches that call. This was the root cause traced across 7 forced logouts (BOB-116/120/123/124/125/126): a `pytest` runâ€™s `kill(os.getpgid(proc.pid), SIGKILL)` resolved `proc.pid` to a `MagicMock`, `os.getpgid()` (or the mock itself) yielded `1`, and the kill call became a broadcast kill of the operatorâ€™s own session.

The fix landed as a guard requiring the id to be a real `int` greater than `1` immediately before the dangerous call (`download-proxy/src/merge_service/search.py:1200-1240`, two identical guards on `_pid` and `_pgid`). This gate makes that shape a **mechanically enforced invariant** instead of tribal knowledge living only in a comment â€” per Â§11.4.226 (evidence-class-at-closure), a source-only fix with no standing guard is exactly the class of closure that silently reopens the next time someone touches the cleanup path.

Usage

```
# Default scope: download-proxy/src/, plugins/, scripts/ (relative to the
# repo root this script resolves from its own location).
scripts/pre_build/check_cm_killpg_pgid_guard.sh

# Explicit-path mode â€” scan exactly the given file(s)/directory(ies)
# instead of the default scope (directories are still walked with the
# same excludes). Used by the meta-test to run against hermetic fixtures.
scripts/pre_build/check_cm_killpg_pgid_guard.sh /path/to/file.py
scripts/pre_build/check_cm_killpg_pgid_guard.sh /path/to/fixture/dir

# Verbose (prints every hit that WAS correctly guarded, not just findings)
scripts/pre_build/check_cm_killpg_pgid_guard.sh -v

scripts/pre_build/check_cm_killpg_pgid_guard.sh --help
```

Inputs

- **Positional arguments** (optional): zero or more file/directory paths.
  - Zero arguments â†’ default scope: `download-proxy/src/`, `plugins/`, `scripts/` under the repository root, walked recursively.
  - One or more arguments â†’ scan exactly those paths instead (single files must end in `.py` or `.sh` to be considered; directories are walked with the same directory-name excludes as the default scope).
- **`-v` / `--verbose`**: also print each hit that passed the guard check.
- **`-h` / `--help`**: print the in-source documentation header and exit 0.
- No environment variables are read.

Outputs

- **stdout**: a run header (scope, window size, file count) and either `PASS: CM-KILLPG-PGID-GUARD clean across N file(s)` or (on `-v`) a `guarded: <path>:<line> [identifier=...]` line per correctly-guarded hit.
- **stderr** (only on failure): a `=== FINDINGS ===` block, one `FAIL: <path>:<line>: unguarded <call kind> â€” <trimmed source> [identifier=<ident-or-none>]` line per unguarded hit, followed by a `FAIL: <N> unguarded killpg/kill-group hit(s) ...` summary line.
- **Exit codes**:
  - `0` â€” zero unguarded hits (or, in explicit-path mode, zero matching files â€” an empty scope is not itself a finding).
  - `1` â€” one or more unguarded hits; see stderr for the per-hit report.
  - `2` â€” usage/environment error (unknown flag, or an explicit PATH argument that does not exist). Distinct from finding-level FAIL, per Â§11.4.201(4) â€œconservative-safe default on an unresolvable signalâ€.

Detection patterns

Language	Patterns
Python (`*.py`)	`os\.killpg\(` Â· `os\.kill\(\s*-`
Bash (`*.sh`)	`killpg\s` Â· `kill\s+-[1-9][0-9]*\b`

Two deliberate scoping decisions, both evidence-based (Â§11.4.6), not guesses:

1. **Comment-only lines are never hit candidates.** A line that is entirely a comment (starts with #, after optional leading whitespace) is skipped before pattern matching. `search.py` itself contains two comment lines that quote the dangerous call syntax while *documenting* the BOB-126 fix (`# \os.killpg(1, sig)` == `kill(-1, sig)` under glibc`) â€” those are text, not code, and flagging them would be a false-positive refusal on the very comment explaining the fix (Â§11.4.201).
2. **The bash signal-kill pattern excludes signal `0`** (requires `[1-9][0-9]*`, not the briefâ€™s literal `\d`). `kill -0 PID` delivers no signal at all (see `kill(2)`) â€” it is the standard POSIX liveness-probe idiom (â€œis this process still alive?â€�), used unguarded twice in this repo today (`scripts/system-slice-watchdog/user1000-watchdog.sh`, `scripts/tunnel-keepalive.sh`) and structurally incapable of ever becoming the BOB-126 broadcast-kill defect, since it never delivers a real signal regardless of what pid/pgid it targets. Including `-0` would make the gate FAIL against the real tree on two known-safe call sites â€” itself a Â§11.4.201 FAIL-bluff (a false-positive refusal is exactly as forbidden as a false-negative pass).

Guard recognition

For each real (non-comment) hit, the target identifier is extracted from the call itself â€” the first argument to `killpg(`, the name after the leading `-` in `kill(-`, or the token following `killpg` / the signal-kill call in bash. The 10 lines immediately preceding the hit are then checked for **both**, on the same identifier, anywhere in that window (same line or split across lines):

- `isinstance(<ident>, int)`
- `<ident> > 1` â€” the literal digit `1`; `> 0`, `> 10`, `> 100` do **not** satisfy this (a pgid/pid of `1` is precisely the broadcast-kill value â€” the whole point of the guard is excluding it).

When the callâ€™s argument is not a simple identifier (e.g.Â a nested call expression such as `os.killpg(get_pgid(), sig)`), the identifier cannot be extracted and the check falls back to the looser, non-identifier-scoped form of the same two conditions (any `isinstance(..., int)` and any `> 1` in the window) rather than silently passing.

Self-exclusion (anti-carrier guard)

This gateâ€™s own source necessarily quotes the very patterns it detects (the pattern strings themselves, the report label text, the doc comments describing the bash `kill -SIG` idiom). Its own file is therefore **structurally excluded from its own scan** â€” resolved by absolute path, not by careful wording â€” closing the Â§11.4.196(D)/Â§12.12 â€œinstrument matches its own carrier textâ€ footgun class. Verified in-session: without the exclusion, the gateâ€™s own comments trigger `killpg\s` four times (the word â€œkillpgâ€ followed by a space occurs in prose describing the tool).

Side-effects

None. The gate is read-only: it never modifies any scanned file. It creates one `mktemp` findings file for its own run and removes it on exit via `trap ... EXIT`.

Dependencies

- `bash` (uses `[[ ]]`, `<<<` here-strings, process substitution).
- `grep -E`, `sed -E`, `find` â€” all standard on the projectâ€™s supported hosts; no GNU-specific extension is required beyond `\s`/`\d`-style shorthand support in `grep -E` (present on both GNU grep and the `ugrep`-as-`grep` compatibility layer this host ships).
- No Python, no network access, no repository write access.

Cross-references

- **BOB-126** â€” the forced-logout incident chain (BOB-116/120/123/124/125/126) this gateâ€™s real-tree pattern was extracted from.
- **Â§11.4.238** â€” QA-discovery-channel mandate: this gate is the â€œnew or strengthened automated checkâ€ a coverage-escape audit requires for a defect first found out-of-band.
- **Â§11.4.226** â€” evidence-class-at-closure: a standing guard, not merely a source-side comment, is what keeps a fix from silently reopening.
- **Â§11.4.115 / Â§11.4.135** â€” RED-first meta-test discipline / permanent regression-guard suite: `tests/pre_build/test_check_cm_killpg_pgid_guard.sh` is this gateâ€™s paired Â§1.1 mutation harness.
- **Â§11.4.196(D) / Â§12.12** â€” the â€œinstrument matches its own carrier textâ€ footgun class this gateâ€™s self-exclusion closes.
- **Â§11.4.201** â€” every guard/gate asserts the real condition; a false-positive refusal is a FAIL-bluff exactly as forbidden as a false-negative pass (the rationale for excluding comment-only lines and the `kill -0` liveness-probe idiom).
- `download-proxy/src/merge_service/search.py:1200-1240` â€” the real, currently-guarded call sites this gate verifies stay guarded.

Companion files

- `scripts/pre_build/check_cm_killpg_pgid_guard.sh` â€” the gate itself.
- `tests/pre_build/test_check_cm_killpg_pgid_guard.sh` â€” the Â§1.1 paired meta-test: one golden-good fixture (properly guarded), two golden-bad fixtures (no guard at all; guard present but checks `> 0` instead of `> 1`), plus a real-tree smoke run against the current checkout.

Last verified

2026-08-19 â€” `bash tests/pre_build/test_check_cm_killpg_pgid_guard.sh` ran GREEN (4/4: golden-good rc=0, golden-bad-1 rc=1, golden-bad-2 rc=1, real-tree default scope rc=0); `shellcheck -x` clean on both scripts with zero findings.